Python: str, list, set, dict, tuple + regex (re) — A4

Strings (str)	
`s.strip() / lstrip() / rstrip()`	Trim whitespace/chars
`s.lower() / upper() / title()`	Case transforms
`s.split(",") / rsplit()`	Split to list
`",".join(parts)`	List to string
`s.replace(old, new)`	Replace substring
`s.startswith() / endswith()`	Prefix/suffix check
`s.find() / rfind()`	Index or -1
`s.count(sub)`	Count occurrences
`s.isdigit() / isalpha() / isalnum()`	Char-type checks
`s.partition(sep)`	Head, sep, tail
`s.removeprefix() / removesuffix()`	Py3.9+ trim affix
`s.zfill(n) / ljust() / rjust()`	Pad / align
`f"{x}" / format()`	Interpolation
`s[1:4] / s[::-1]`	Slice / reverse

Tuples (tuple)	
`t.count(x) / index(x)`	Only tuple methods
`a, b, c = t`	Unpack
`a, *mid, z = t`	Star unpack
`(x,) single`	One-element tuple
`t1 + t2`	Concatenate
`x in t`	Membership
`tuple(iterable)`	Construct
`d[(region, sku)]`	Composite dict key
`return a, b`	Multi return
`sorted(t)`	Sort → list
`hash(t) if no mutables`	Hashable record

Sets (set)	
`s.add(x)`	One element
`s.update(iterable)`	Many elements
`s.remove(x) / discard(x)`	Remove (error / safe)
`s.pop() / clear()`	Arbitrary pop / empty
`a | b`	Union
`a & b`	Intersection
`a - b`	Difference
`a ^ b`	Symmetric difference
`x in s`	O(1) membership
`s.issubset(t) / issuperset()`	Containment
`s.isdisjoint(t)`	No overlap
`set(iterable)`	Dedupe (unordered)
`frozenset(s)`	Immutable set
`{x for x in it if cond}`	Set comprehension

Lists (list)	
`lst.append(x)`	Add end
`lst.extend(iterable)`	Add many
`lst.insert(i, x)`	Insert at i
`lst.pop([i])`	Remove & return
`lst.remove(x)`	Remove first x
`lst.clear()`	Empty list
`lst.sort(key=fn, reverse=)`	Sort in-place
`sorted(lst)`	New sorted list
`lst.reverse()`	Reverse in-place
`lst.index(x) / count(x)`	Find / count
`lst.copy() / lst[:]`	Shallow copy
`[x*2 for x in lst]`	List comprehension
`[x for x in lst if cond]`	Filter comp
`list(map(fn, lst))`	Transform
`list(filter(pred, lst))`	Filter iterator

Dictionaries (dict)	
`d[key] = val`	Assign
`d.get(k, default)`	Safe read
`d.setdefault(k, default)`	Get or set
`d.keys() / values() / items()`	Views
`d.update(other)`	Merge in-place
`d | other / |=`	Merge Py3.9+
`d.pop(k) / popitem()`	Remove pair
`del d[k] / clear()`	Delete / empty
`k in d`	Key membership
`dict.fromkeys(keys, v)`	Init keys
`{k: f(v) for k,v in d.items()}`	Dict comp
`{r['id']: r for r in rows}`	Lookup table
`copy() / deepcopy()`	Shallow / deep

Regex (re module)	
`re.match(pat, s)`	Start of string
`re.search(pat, s)`	First anywhere
`re.findall(pat, s)`	All matches
`re.sub(pat, repl, s)`	Replace
`re.split(pat, s)`	Split on pattern
`re.compile(pat)`	Reuse pattern
`m.group() / group(1)`	Whole / group
`m.groups()`	All groups tuple
`r"\d+" raw`	Raw string for \d
`. ^ $ \b`	Any, start, end, word
`\d \w \s \D \W \S`	Digit, word, space
`+ * ? {n,m}`	Quantifiers
`[abc] [a-z] [^0-9]`	Character class
`( ) | (?: )`	Group / alt / non-cap
`*? +?`	Non-greedy

~80 ops • str, tuple, set | list, dict, regex • A4 • DE-focused